



**WEST UNIVERSITY OF TIMIȘOARA
FACULTY OF COMPUTER SCIENCE
MASTER STUDY PROGRAM: CYBERSECURITY**

MASTER THESIS

SUPERVISOR:
Lect. Dr. Ing. Darius Galis

GRADUATE:
Paul - Andrei Abrudan

**TIMIȘOARA
2026**

WEST UNIVERSITY OF TIMIȘOARA
FACULTY OF COMPUTER SCIENCE
MASTER STUDY PROGRAM: CYBERSECURITY

Automated Security Analysis of the Rust Crate Ecosystem: A Distributed Framework for Vulnerability and Malicious Code Detection

SUPERVISOR:
Dr. Lect. Darius Galis

GRADUATE:
Paul - Andrei Abrudan

TIMIȘOARA
2026

Abstract

The Rust programming language has gained popularity in multiple programming fields due to its memory safety features and performance. Rust's package ecosystem, crates.io, delivers over 150.000 crates to the language's users. Due to the rapid growth of the language and ecosystem, potential security risks are present, including known vulnerabilities in dependencies, a history of supply chain attacks through code injection and typosquatting, secrets being leaked in source code and cases of malicious build scripts present in the optional build.rs file at the root of the packages. This thesis presents a distributed framework for automating security analysis on the Rust package library ecosystem. The system parses metadata from the crates.io database dump, constructing a dependency graph and applying multiple analysis modules: Vulnerability scanning using 'cargo audit', which uses the RustSec Advisory Database, Secret leakage detection using the FOSS tool GitLeaks, heuristic analysis for the 'build.rs' script, detecting network calls, process creation, obfuscation and suspicious TLDs used, the possibility of LLM code scoring using LM Studio's REST API, typosquatting detection using multiple string similarity algorithms and transitive dependency chain risk propagation. Horizontal scaling is possible across multiple nodes, achieved with Redis used as a work queue, with the possibility of upgrading to a faster in-memory database. The results are persisted in a PostgreSQL database and can be visualized through the web dashboard. This thesis aims to make it easier to search for cases of malicious dependencies in the Rust ecosystem, providing tools for security auditing.

Keywords: crates.io, Rust, supply chain security, typosquatting, distributed systems, software security

Statement of Scientific integrity

I declare that this BSc / MSc thesis is an original report and represents my own work, conducted under supervision of [name(s) supervisor(s)]. All research methods, data collection, analysis and interpretation presented in this thesis are accurately reported and all sources have been cited appropriately where I have made use of ideas and/or visuals of others.

If you did not use generative AI tools: I did not use generative AI during the writing and research process.

If you used generative AI tools: I used generative AI during the writing and research process. The details are in the Appendix F.

Contents

1	Introduction	9
1.1	Context and Motivation	9
1.2	Problem Statement	10
1.3	Objectives	10
1.4	Scope and Delimitations	10
1.5	Thesis Structure	11
2	State of the art	13
2.1	Software Supply Chain Security	13
2.1.1	Supply Chain Incidents	13
2.1.2	Classification of Supply Chain Attacks	15
2.1.3	The Dependency Problem at Scale	16
2.1.4	Supply chain frameworks	17
2.2	Package Ecosystem Security Analysis	17
2.2.1	Vulnerability Databases and Advisory Systems	17
2.2.2	Software Bill of Materials	19
2.2.3	Large-Scale Ecosystem Security Studies	19
2.3	Typosquatting in Package Registries	20
2.3.1	String Similarity Algorithms for Detection	21
2.3.2	Typosquatting in the Rust Ecosystem	21
2.4	Malicious Code Detection in Package Ecosystems	21
2.4.1	Categories of Malicious Behavior	21
2.4.2	Static Analysis Approaches	22
2.4.3	Dynamic Analysis Approaches	22
2.4.4	Machine Learning Approaches	23
2.5	Build Script Security Risks and Detection	23
2.6	Large Language Models for Code Analysis	24
2.6.1	Evolution of Code-Aware Language Models	24
2.6.2	LLMs for Vulnerability Detection	24
2.6.3	Challenges in Large-Scale LLM Security Analysis	25
2.6.4	Local vs. API-Based LLM Deployment	25
2.7	Secret Detection in Source Code	26
2.7.1	The Problem of Hardcoded Secrets	26
2.7.2	Secret Detection Approaches	26
2.7.3	Gitleaks	26
2.7.4	Secrets in the Rust Ecosystem	26
2.8	Existing Tools and Related Work	26
2.8.1	Rust-Specific Security Tools	26

2.8.2	General-Purpose Security Tools	27
2.8.3	Commercial and Community Supply Chain Security Platforms	27
2.8.4	Academic Research on Ecosystem Security	27
2.9	Distributed Systems for Large-Scale Analysis	28
2.9.1	Challenges of Ecosystem-Scale Analysis	28
2.9.2	Work Queue Architectures	28
2.10	Summary and Research Gap	28
2.10.1	Ecosystem-Specific Gaps for Rust	29
2.10.2	Methodological Gaps	29
2.10.3	Contributions of This Thesis	29
3	Methodology	31
3.1	Research Approach	31
3.2	Threat Model	31
3.2.1	System Model	31
3.2.2	Attacker Model	31
3.2.3	Attacker Classes	32
3.2.4	Attack Vectors Addressed	32
3.2.5	Scope and Assumptions	32
3.3	Data Collection Strategy	33
3.4	Analysis Framework Design	33
3.5	Evaluation Metrics	33
4	System Architecture	35
4.1	High-Level Architecture Overview	35
4.2	Data Ingestion Pipeline	36
4.3	Dependency Graph Construction	36
4.4	Distributed Work Queue	36
4.5	Database Schema Design	36
4.6	Web Dashboard	37
4.7	Deployment and Infrastructure	38
5	Analysis Modules	39
5.1	Cargo Audit - Vulnerability Scanning	39
5.2	Gitleaks - Secret Detection	39
5.3	Build Script Analysis	39
5.4	LLM-Assisted Malicious Code Detection	40
5.5	Typosquatting Detection	40
5.6	Transitive Dependency Risk Propagation	41
6	Implementation Details	43
6.1	Technology Stack	44
6.2	CLI Application Structure	45
6.3	Redis Queue Implementation	45
6.4	Cycle Detection and Resolution	45
6.5	Worker Lifecycle and Fault Tolerance	45
6.6	Batch Processing and Database Optimization	46

7	Results and Evaluation	47
7.1	Dataset Overview	47
7.2	Vulnerability Landscape	48
7.3	Dependency network	49
7.4	Suspicious <code>build.rs</code> Analysis	51
7.5	Temporal Trends	51
8	Discussion	55
8.1	Comparison with Existing Approaches	55
8.2	Limitations	55
8.2.1	Detection Limitations	55
8.2.2	LLM-Specific Limitations	55
8.2.3	Scalability and Resource Limitations	56
8.2.4	Ground Truth Limitations	56
8.2.5	Generalizability Limitations	56
8.3	Threats to Validity	56
9	Conclusions and Future Work	57
9.1	Summary of Contributions	57
9.2	Future Directions	58
	Bibliography	61
A	Database Schema	63
B	Configuration Reference	65
C	Web Dashboard	67
D	Artifact Documentation and Reproducibility	69
D.1	Software Artifacts	69
D.2	Data Artifacts	69
D.3	Hardware Requirements	69
D.4	Software Requirements	70
D.5	Reproduction Steps	70
D.6	Expected Outputs	71
E	Glossary	73
F	Generative AI Tools Usage	75
F.1	Claude Opus 4.6, DeepSeek V4 Pro — Rephrasing, Typo Checking, and Bibliography	75
F.2	Code Generation Assistance	76

Chapter 1

Introduction

1.1 Context and Motivation

All relatively modern software relies on thirdparty packages, distributed through different centralized registries. The Rust programming language has grown from its initial release by Mozilla back in 2015, to today where crates.io hosts over 150.000 crates with multiple billions of downloads, being the pillar of the Rust library supply chain.

This growth can be a double-edged sword, while being beneficial for productivity, it introduces an immense attack surface, which remains underexplored as of now. In most languages, including Rust, when a developer adds a dependency to their project, they not only trust the direct dependency but also all its entire dependency tree, which can contain hundreds if not thousands of packages. A single compromised dependency, whether due to an unpatched vulnerability, a typosquat or due to an adversary takeover, can propagate malicious code across a wide part of the ecosystem.

Relatively recent incidents in other package ecosystems, such as npm's attack in 2018 known as 'event-stream', 'ua-parser-js' from 2021 and the 'colors.js' and 'faker.js' sabotages in 2022, demonstrated that supply chain attacks are practical attacks, very effective and can become extremely complex. Rust's memory safety guarantees mitigate some classes of vulnerabilities (such as buffer overflows), they provide no help against attacks such as memory-safe malicious code and secret exfiltration from the environment.

Another attack vector specific to some languages, including Rust, is that the build system allows arbitrary code execution by crates at compile time, using 'build.rs' scripts. The scripts can perform operations such as network communication, process execution and have file system access. This mechanism is essential for some use cases such as native library linking and code generation, however it also is a dangerous attack vector that can infect a machine just compiling a malicious crate, without ever executing the resulted binary.

The motivation comes from the observation that some security tools already exist for specific Rust crate analysis, however no ecosystem-wide comprehensive framework exists, that integrates multiple modules and can apply them at scale.

1.2 Problem Statement

Rust’s crate ecosystem, lacks a scalable analysis framework that is capable of performing module-based scanning, first, the scale of crates.io is massive, with over 150.000+ crates, any ecosystem-wide analysis requires a scalable and distributed architecture, capable of processing crates efficiently. Individual tools address specific parts of security analysis, such as secrets scanning and existing known vulnerabilities. There is no existing solution that combines build script analysis, malicious code detection, typosquatting crate name analysis, vulnerability scanning and secret detection into a complete comprehensive solution. It is also the case that a crate with no direct vulnerabilities can still pose a risk if it exports an API dependent on a vulnerable package, tracking these risks is still an existing challenge. The ‘build.rs’ script allows arbitrary code execution at compile time and it is the case that no study has tracked possibly suspicious build script behaviors across the entire crates.io database. While typosquatting has been studied in other package libraries such as npm and PyPI, the Rust’s ecosystem has seen no studies regarding this. Large Language Models have demonstrated capabilities in understanding code, however the deployment of LLM analysis at scale has not been explored over package libraries.

1.3 Objectives

The objective of this thesis is to design, implement and evaluate a toolkit facilitating comprehensive distributed security analysis over the Rust’s crate ecosystem, the specific objectives are:

Designing a modular toolkit, which unifies multiple modules and analysis techniques into a single pipeline, enabling user friendly security assessment of crates published on crates.io Implementing distributed processing over the entire toolkit, using a ”in-memory database”-based work queue, together with topological ordering of the crates dependencies, we enable horizontal scaling across workers and achieving acceptable throughput on consumer hardware. Develop and integrate multiple analysis modules such as: vulnerability scanning using RustSec’s Database, secret detection in available source code, heuristic ‘build.rs’ analysis for detecting unusual behaviours such as network communication, process execution, obfuscated/encrypted code and suspicious TLD usage, LLM-based malicious code scoring, typosquatting detection using multiple similarity algorithms (Levenshtein, Damerau-Levenshtein, Jaro-Winkler, keyboard proximity distance, and prefix), and dependency chain propagation analysis. Conduct an entire ecosystem-wide analysis, either by running the framework on a subset of the crates.io registry, or the entire database, characterizing and doing manual analysis on provided warnings. Develop a web-based dashboard for user friendly exploration and visualization of analysis results. Evaluate the framework’s performance and scalability, measuring throughput, bottlenecks and possible improvements.

1.4 Scope and Delimitations

In scope:

- Analysis of multiple crates published on crates.io, imported from the official database dump
- Multiple analysis modules as described in the objective list
- Analysis of crate source code from the git repository
- Having distributed processing capabilities using an in-memory database for work distribution and PostgreSQL for resulting data
- A web dashboard for visualization
- Performance evaluation of analysis and of found problems and exploits

Out of scope:

- Dynamic analysis or runtime behavior
- Analysis of compiled binaries
- Manual verification of all results, due to the framework being designed as a screening tool, requiring human validation
- Real-time monitoring
- Analysis of other package ecosystems
- Formal proofs for detection correctness

1.5 Thesis Structure

This thesis is organized in this way:

Chapter 2 - State of the art: Is a review about existing research and literature on software supply chain security, malicious code detection techniques, package vulnerabilities, typosquatting and build scripts used maliciously. It identifies a research gap related to the previously mentioned objectives.

Chapter 3 - Methodology: Describes the research, data collection and importing, framework design, analysis modules and the evaluation metrics used.

Chapter 4 - System architecture: We detail the high level architecture, covering the data importing, dependency graph creation, the design of the work queue, database schema, the dashboard and deployment infrastructure and optimization scripts.

Chapter 5 - Analysis Modules: We detail the analysis modules used, algorithms, heuristics, fault tolerance and optimization.

Chapter 6 - Implementation Details - We present the technology stack, the CLI app usage, the queue implementation and database implementation

Chapter 7 - Results & Evaluation - This chapter is about the findings resulted by running the framework, vulnerability, leaked secrets, build system statistics, typosquatting possibilities, dependency chain risks and performance benchmarks.

Chapter 8 - Discussion: We interpret the results, compare them to existing literature, discuss limitations and address threats to validity.

Chapter 9 - Conclusion & Future Work: We present a summary of the contribution of this thesis and state possible directions for future research or development

Chapter 2

State of the art

2.1 Software Supply Chain Security

Software supply chain security can be considered one of the most critical problems in software engineering, it reshaped how software developers and organizations from the field approach software development, deployment and risk assessment. The concept was introduced by Ken Thompson in the "Reflections on Trusting Trust", where he modified a compiler to insert backdoors in the compiled binary even when the source code was not containing anything malicious. This attack established that the entirety of a software system depends on the unbroken chain of trust[?].

Following Thompson's work, supply chains remained mostly theoretical or small operations, however the landscape changed with the rise of open-source package distributors and their ecosystems. A new attack surface of extreme scale was introduced, modern software extensively relies on third-party libraries and dependencies, dependencies having their own dependencies which creates a complex tree that can be exploited. Popular tools and software include hundreds if not thousands of transitive dependencies, each one of them can be a potential attack vector for malicious code injection, new attacks consisting of typosquatting, malicious build files, code injection and more.

2.1.1 Supply Chain Incidents

Multiple high profile supply chain attacks have been deployed and demonstrated the high risk of this threat vector. Table 2.1 provides an overview of these incidents.

The SolarWinds attack in 2020 is a state sponsored attack, which compromised the build system of SolarWinds network, by inserting a complex backdoor called SUNBURST into software updates, the attackers managed to deploy the malware to approximately 18.000 customers, including US government agencies and top grossing companies. The attack managed to stay undetected for over eight months and demonstrated how a single compromised dependency in the chain can provide access to a very large number of dependents.

Besides the SolarWinds attack, there was also the Log4Shell vulnerability, even though this was not a malicious attack, this critical vulnerability present in a extremely popular Java library affected millions of applications and threw the industry into chaos for multiple days, requiring maintainers and organization to move quickly and patch their software.

Table 2.1: Summary of Landmark Supply Chain Incidents

Incident	Year	Type	Impact
SolarWinds	2020	State-sponsored	~18,000 customers including US government
Log4Shell	2021	Vulnerability	Millions of Java applications
event-stream	2018	Account compromise	Cryptocurrency theft via Copay wallet
ua-parser-js	2021	Account compromise	8M weekly downloads; crypto miners
xz-utils	2024	Long-term infiltration	SSH backdoor targeting Linux systems

The event-stream incident from 2018 is another example of a supply chain attack, where an attacker gained access to a maintainer account for the package 'event-stream', which is a popular npm package with over 2 million weekly downloads. Social engineering was used together with code injection targeting a cryptocurrency wallet called 'Copay Bitcoin wallet', with the goal being to steal cryptocurrency. The total amount of stolen money is unknown, and even though it is thought to be a small amount, this attack demonstrated yet again how supply chain attacks can be deployed.

In 2021, 'ua-parser-js', which is another popular npm package with approximately 8 million weekly downloads, was compromised in a similar way to the event-stream attack, by gaining access to a maintainer account the adversaries injected cryptocurrency miners and stealers, targeting users and organizations including Microsoft, Amazon and Meta.

A more recent attack, from 2024, is the 'xz-utils' backdoor, where an adversary spent multiple years building trust by contributing to the library, before inserting a complex backdoor in the test files and build system targeting SSH on Linux system, this attack was complex and it contained multiple accounts mentally draining the maintainer. The backdoor was discovered by chance, luckily before being merged into major Linux distros, by Andres Freund, a software engineer working at Microsoft, by observing that on a Debian system ssh logins were unusually slow compared to previous versions, OpenSSH is interacting with a part of xz-utils.

The Rust ecosystem is still developing compared to other ecosystems like npm and PyPI, however it still suffered from multiple supply chain attacks. Table 2.2 summarizes these incidents.

Table 2.2: Rust-Specific Supply Chain Incidents

Incident	Year	Attack Type	Target
CrateDepression	2022	Typosquatting	GitLab CI pipelines
faster_log / async_println	2025	Combosquatting	Cryptocurrency wallet keys
CI/CD targeting crates	Various	Env. exfiltration	GitHub/GitLab CI secrets

In 2022, researchers from SentinelOne discovered a supply chain attack which they named CrateDepression, the attack targeted GitLab CI pipelines through a typosquatting attack, by publishing a crate named 'rustdecimal', which is meant

to be confused with `'rust_decimal'`, which is a legitimate crate providing decimal number support without rounding errors compared to Rust's native floating point types (f32 and f64). The attack detected if the crate was used in a GitLab CI pipeline and would download and execute a binary written in Go, known as a language commonly used for malware since some antimalware solutions struggle to analyze newer languages, and Go is mature enough to be used for complex malware. This attack exfiltrated secrets from the environment and was able to persist in the system by modifying a configuration YAML file. Attacks like this one demonstrate how Rust's memory safety and other security features are unable to protect against these kinds of attacks.

A more recent attack, from 2025, consists of multiple malicious crates including `'faster_log'` and `'async_println'`, which was one of the motivations of this paper. These crates that targeted cryptocurrency systems developers contained code which exfiltrated Solana and Ethereum wallet keys from popular wallets present in the system by scanning for files and environment variables and sending the found data to C2 servers. This is not an example of typosquatting, since there are no similar legit crates, however they promised improved speed compared to the log crate and the println macro from the Rust standard library. These attacks were probably meant to be large-scale due to Rust being popular in cryptocurrency development, with projects including Solana being written in Rust.

Other attacks which are oriented on CI/CD exfiltration, where multiple Rust crates were identified to scan for environment variables related to CI/CD including `'CI'`, `'GITHUB_ACTION'`, `'GITLAB_ID'` and others. These crates would exfiltrate environment files, secrets from the workflow and other credentials.

Multiple campaigns have been identified by security researchers, having dormant crates which contained obfuscated malicious payloads which would be executed at a later time when specific conditions were met.

2.1.2 Classification of Supply Chain Attacks

Several efforts were made by researchers to categorize supply chain attacks to have a deeper understanding of vectors and defensive techniques, which include:

Ohm et al. (2020) “Backstabber’s Knife Collection” [OPSM20] This is a complex review of supply chain attacks which introduced a taxonomy to have them classified:

Typosquatting consists of publishing packages with similar names to popular packages, to deceive developers into installing them instead of the proper ones.

Dependency confusion exploits some package manager’s resolution to prioritize public packages instead of internally used ones. These packages have the same names as internal packages, where the names could be exfiltrated by leaked source code, leaked documentation, guessing common names or internal information from employees.

Compromised accounts where by taking over legitimate maintainers and developers accounts through social engineering, credential leakage, phishing or

other techniques, adversaries disguise themselves as a trusted contributor and inject malicious code, possibly months down the line after gaining access.

Malicious contribution similar to compromised accounts, but instead of gaining access or taking over an existing account, they submit malicious contributions to the project, sometimes being hidden behind a legitimate possibly large pull request, making use of complexity and human error.

Ownership transfer in some ecosystems it is possible to gain access to a package by being added as a maintainer or requesting ownership transfer for abandoned packages.

Besides introducing a taxonomy, Ohm et al. [OPSM20] also introduced the "Backstabber's Knife Collection" which is a dataset that compiles 174 malicious packages from npm, PyPI and RubyGems, dated from 2015 to 2019. The dataset has been reused multiple times in empirical studies of supply chain attacks and malicious package detection and behavior. There is no comparable dataset for crates.io which is another motivation for our data collection and detection.

Ladisa et al. (2023) "A Taxonomy of Attacks on Open-Source Software Supply Chains" [LPMB23] was published at IEEE S&P 2023. This paper proposed another taxonomy based around the lifecycle of software development:

Development phase attacks targeting local environments, such as IDE plugins, development tools and more.

Build phase attacks which compromise build systems such as CI/CD or build dependencies.

Distribution phase attacks which target package registries, distribution channels and infrastructure and more.

The taxonomy identified over 100 attack vectors across these phases and had defenses mapped to each one of the vectors, which revealed gaps in coverage, especially present in the build phase.

2.1.3 The Dependency Problem at Scale

We can consider the fundamental challenge of supply chain attacks to be the transitive nature of dependencies. Zimmermann et al. [ZSTP19] "Small World with High Risks: A Study of Security Threats in the npm Ecosystem" analyzed the npm ecosystem and revealed the following:

- The average npm package trusts 79 packages.
- Up to 40% of packages depend on at least one package with a public known vulnerability, even if it's exploitable or not in a real world scenario.
- A single compromised maintainer account could affect multiple million packages through transitive dependencies.

The study analyzed over 5.3 million versions, over 199.000 maintainers and over 600 security issues. The study found that on average, installing an npm packages trusts 79 packages and 39 maintainers, and some maintainers or packages can impact large parts of the ecosystem. Even though the study was specific to NPM's architecture, Rust tends to follow a similar culture of micropackages. Rust's culture highly encourages code reuse and the development of small focused crates, similar to the Unix philosophy, which results in complex and deep dependency trees. A typical rust application can have hundreds of transitive dependencies. Our work partially fills this gap by analyzing the dependency graph of crates.io.

Many crates suffer from having a single maintainer, which is almost always an unpaid volunteer. The previously mentioned xz-utils incident demonstrated how attackers are able to mentally drain maintainers with harassment and spend multiple years in gaining trust before executing an attack.

2.1.4 Supply chain frameworks

The supply-chain levels for software artifacts (SLSA) [Ope24] is a framework for securing software supply chains by defining four levels of security. For level 1 it documents the build process, and for levels 2 to 4 it adds cryptographic signing and verification of the software provenance, isolated and reproducible builds and a human review process. SLSA is complementary to supply chain tools such as cargo audit however SLSA focuses on just build integrity and provenance while our framework provides findings that can be used for assessment and mitigations. Most of the Rust's ecosystem currently operates at levels 0 to 1 for the majority of the crates, with some using tools for cryptographic signing but this is limited to a small percentage of the crates.

2.2 Package Ecosystem Security Analysis

2.2.1 Vulnerability Databases and Advisory Systems

The detection and communication of known vulnerabilities is managed through databases maintained by diverse ecosystem-specific organizations, however maintainers are not always properly notified and incentivised to update dependencies.

The RustSec Advisory Database is managed by the Rust Secure Code Working group. It is the source of security advisories for the crates ecosyste, they cover vulnerabilities, including memory safety issues by poor unsafe code, cryptographic problems and logic errors. They include:

- Bugs that violate Rust's safety guarantees, even when not directly exploitable.
- Unmaintained crates which usually accumulate security issues.

Each advisory includes metadata, such as affected versions, patched versions, a scoring system and links to relevant CVEs.

The Common Vulnerability Scoring System (known as CVSS), provides a standardized method for rating vulnerabilities based on severity and impact, the score is from 0.0 to 10.0, being calculated from eight metrics:

Attack vector describes how the vulnerability can be exploited, such as network, adjacent, local and physical.

Attack complexity describes the required conditions for the attack.

Privileges Required describes the level of access the attacker needs.

User Interaction if the attack requires any user interaction such as clicking a button or opening a file.

Scope describes if the problem affects anything outside of the component.

Confidentiality Impact measures the impact on data confidentiality.

Integrity Impact such as data tampering, code execution and others.

Availability Impact such as Denial of Service and others.

The mathematical formula for the score combined these metrics into the score, which is furtherly classified as none, low, medium, high and critical. The formula is as follows:

$$\text{BaseScore} = \begin{cases} \text{Roundup}(\min(\text{Impact} + \text{Exploitability}, 10)) & \text{if Scope} = \text{Unchanged} \\ \text{Roundup}(\min(1.08 \times (\text{Impact} + \text{Exploitability}), 10)) & \text{if Scope} = \text{Changed} \end{cases}$$

where:

$$\text{Impact} = \begin{cases} 6.42 \times \text{Impact}_{\text{base}} & \text{if Scope} = \text{Unchanged} \\ 7.52 \times (\text{Impact}_{\text{base}} - 0.029) - 3.25 \times (\text{Impact}_{\text{base}} - 0.02)^{15} & \text{if Scope} = \text{Changed} \end{cases}$$

$$\text{Impact}_{\text{base}} = 1 - (1 - \text{Conf}) \times (1 - \text{Integ}) \times (1 - \text{Avail})$$

$$\text{Exploitability} = 8.22 \times \text{AttackVector} \times \text{AttackComplexity} \times \text{PrivReq} \times \text{UserInt}$$

Our implementation does implement this algorithm, correctly parsing the CVSS string and scaling it from 0 to 100.

Cargo audit is the official tool for checking a project's dependencies against the RustSec database. Using a command line interface, it includes features such as:

- JSON output for automation and further integration.
- Warnings for unmaintained crates.

2.2.2 Software Bill of Materials

A software bill of materials (SBOM) is defined as an inventory of all components, dependencies and metadata of a project, it is intended to improve vulnerability management across the supply chain. U.S. Executive Order 14028 and CISA’s SBOM guidance treat SBOM as a baseline requirement. Rust ecosystem has tools like cargo-sbom for SPDX SBOM [Lin24b] and cargo-cyclonedx for CycloneDX SBOM [OWA24], however the ecosystem still has a limited adoption and no standardization of their usage or the format used.

Due to the lack of SBOM adoption in this ecosystem, immature tooling and incomplete coverage of all dependencies we do not consider SBOMs to be practical for the implementation in this framework.

2.2.3 Large-Scale Ecosystem Security Studies

Multiple research efforts were already conducted on some package ecosystems, however the Rust ecosystem is still underexplored as it is still in its early stages compared to ecosystems like npm and PyPI, which have been studied.

The npm ecosystem has been studied in multiple papers:

- Staicu and Pradel (2018) analyzed over 235.000 npm packages for injection vulnerabilities, finding thousands of exploitable packages.
- Zahan et al. (2022) examined the relationship between maintainer activity, contributor diversity and security findings.
- Lauinger et al. (2017) analyzed client-side libraries in websites and found extremely high usage of outdated and vulnerable versions.

The PyPI has also received attention from researchers:

- Vu et al. (2020) documented typosquatting attacks in PyPI, which proposed detections.
- Guo et al. (2023) analyzed malicious package campaigns and investigated evasion techniques.
- PyPI infrastructure has implemented complex malware detection including YARA rules and behavior analysis.

Cross-ecosystem studies have also been done:

- Decan et al. (2018) compared npm, CRAN and RubyGems
- Ponta et al. (2020) compared patterns between Maven, npm and PyPI.

PyPI has invested in infrastructure focused on malware detection and telemetry. Livehaul is used to collect download telemetry to detect anomalies. For malware detection specifically, YARA rules are used to scan for known malicious patterns [Pyt23].

Besides academic research, Sonatype has an annual report called ”State of the Software Supply Chain” [Son26] which provides large-scale analysis across multiple ecosystems. The report from 2024 analyzes more than 7.000.000 open source

projects and 6.6 trillion downloads. A 156% YoY increase was found in malicious packages. It showed that 95% of the time when a version with a fixed vulnerability is released, 80% of the dependents remain on the older version for over a year. The report shows a consistent gap between fixes and patching.

However, despite Rust's high growth and importance in the industry, including security critical fields, there is a lack of a comprehensive security analysis of crates.io, existing research is mainly focused on memory safety and other language features, but not on the entire ecosystem. This thesis aims to address this gap.

2.3 Typosquatting in Package Registries

Typosquatting is a form of attack where adversaries register malicious packages with names similar to popular packages, usually containing just a typo, this is to exploit human error, in case developers mistype packages when adding dependencies.

A possible threat model could be:

Attacker capabilities Register arbitrary package names, publish malicious code and track downloads.

Attack surface All packages installation events (manual editing of Cargo.toml, cargo add or automated tools).

Victim behavior Occasional human errors, poor verification of dependencies, reliance on ai autocomplete.

Attacker goals Code execution, which can lead to credential theft, cryptocurrency theft and mining, further remove code execution and more.

Researchers have identified several patterns used in typosquatting attacks, such as:

Character substitution Replacing characters with similar alternatives, such as 0 instead of o, l instead of 1, l instead of i and others. This is not as common due to user's not usually copy-pasting package names, and being less effective on getting users to install the dependencies.

Character omission By removing a single character, such as 'rust_decimal' being typosquatted by 'rustdecimal', this being a previously mentioned attack.

Character addition Similar to character omission, but instead of removing a character another one is added.

Character transposition by swapping adjacent characters, such as 'tokoi' instead of 'tokio', this is another easily spotted pattern, however might get some users due to users typing behavior.

Separator manipulation by changing, adding or removing separators, such as 'serde-json' vs 'serde_json', this is a common pattern in the rust ecosystem due to the common use of underscores and dashes.

Prefix and suffix manipulation by removing or adding prefixes and suffixes, usually common ones for the package ecosystem, such as 'rust-serde' vs 'serde'.

Combosquatting which is combining a legitimate name with commonly used modifiers, such as ‘faster_log’ and ‘async_println’ which are previously mentioned.

2.3.1 String Similarity Algorithms for Detection

Typosquatting detection mostly relies on calculating the similarity between multiple package names using different algorithms, such as:

Levenshtein Distance also known as edit distance, which counts the minimum single character operations to modify a script to another.

Damerau-Levenshtein Distance which is an extension of the Levenshtein distance, which adds transposition of adjacent characters, this is useful because of the keyboard layout permitting these kinds of errors.

Jaro-Winkler similarity it computes the number of matching characters, number of transpositions and computes a score combining these factors, after this it adds extra weight to some common prefixes.

Keyboard proximity distance which measures the physical distance between keys on a keyboard, usually QWERTY because of its dominance.

Phonetic similarity there are algorithms like soundex or metaspone which compare how names sound rather than their spelling, however this is not very useful for typosquatting detection, since usually package names are transmitted through text, not audio.

There is also the possibility of running all of these algorithms and computing a meta-score based on weights on the previously mentioned algorithms or others.

2.3.2 Typosquatting in the Rust Ecosystem

Compared to other ecosystems, Rust’s crates.io has a more specific naming pattern that affects typosquatting attacks and detection. There is a heavy use of hyphens and underscores, together with common suffixes like ‘-rs’ and others.

Rust Foundation Typosquatting Protection The Rust Foundation has implemented typosquatting detection using the crate ‘typomania’, it computes similarity scores for new crates against existing popular ones, and flags names very similar to current popular crates, requiring manual approval. It is confirmed that this system has successfully prevented multiple typosquatting attacks.

2.4 Malicious Code Detection in Package Ecosystems

2.4.1 Categories of Malicious Behavior

Malicious code from package ecosystem exhibits pattern such as the following:

Data exfiltration, which involves stealing information from the victim system, such as environment variables (API keys, credentials, database connection strings, sensitive settings), files (configuration files, SSH keys, cryptocurrency wallet keys, sensitive configuration files), browser data, clipboard contents and more. The crates previously mentioned that targeted Solana and Ethereum users fit into this category, searching for cryptocurrency wallets and private keys.

Reverse shells and backdoors, establishing persistent access to the victim system.

Cryptocurrency mining, mostly CPU-based mining of Monero due to its privacy and hard to scale mining algorithm.

Payload downloading, by downloading additional malware. It sometimes uses legitimate services as the hosts such as github, free CDNs, pastebin or others.

Conditional execution, sometimes, the malware authors implement malicious behavior execution only in some branches of the code, ran only if specific conditions are met, such as a CI environment being detected, executing only in specific countries or after a specified date.

2.4.2 Static Analysis Approaches

Static analysis is analysis of a binary or code without executing it, it is very scalable compared to hybrid analysis but more limited in its capabilities.

Signature-based detection relies on known malicious patterns such as having hardcoded specific IP addresses and domain names, characteristic code from known malware or specific obfuscation patterns used in previous malware.

YARA rules are widely used for signature-based detection, they allow us to define specific, even complex patterns combining features such as string matches, byte patterns and metadata.

Behavioral pattern matching looks at more complex patterns, such as sequences of calls, network connections from unexpected modules, file system operations on unusual files and more and more.

High entropy in code can also indicate either encrypted or compressed code, encoded or encrypted strings, large embedded binary data and more, usually not used in benign packages.

Dependency analysis is used to examine which dependencies a package has, we can identify if there are unexpected network calls or dependencies in a package that should not include them, having other dependencies on packages with known security issues or unusual version requirements, such as requiring an extremely old version of a package.

Metadata analysis, looking at new maintainers, recent ownership transfers, unusual releases such as multiple versions in a rapid succession, rewrites of old commits, changing version tag pointers to other commits, and others.

2.4.3 Dynamic Analysis Approaches

Dynamic analysis is the counter part to static analysis, that involves emulating the code or executing it in a sandboxed or controlled environment to observe the runtime behavior.

Sandboxed execution is running packages in isolated environments such as virtual machines or containers, with the interception of system calls, network connec-

tions and more to assess the behavior of the package.

Behavioral monitoring is used to record network connections, file accesses, process execution, environment variable access and more.

Differential analysis refers to comparing behavior between two versions, to check if there is any change in behavior such as new network connections to exiting or new servers and other different patterns.

2.4.4 Machine Learning Approaches

Feature based classification consists of extracting features such as metadata from packages and training machine learning classifiers. It can be effective for some patterns. Code metrics, dependency graph, metadata and behaviors are analyzed.

Text based classification is ran on the source code and applies NLP techniques such as bag of words, TF-IDF, code embeddings, sequence models and more to classify code.

Graph neural networks model dependencies as graphs and are also used for malicious code detection by analyzing for suspicious patterns.

Machine learning approaches can be effective, however they face challenges such as class imbalance, concept drift, adversarial evasion especially for text based classification and LLMs.

2.5 Build Script Security Risks and Detection

Rust's `build.rs` feature is very powerful and allows crates to execute arbitrary Rust code at the compile time of a package and its dependents, cargo first checks for a file named `'build.rs'` in the root directory of a crate, compiles it's code and executes it before compiling the main crate. This allows the build scripts to perform advanced and complex operations and possibly infect systems.

There are multiple legitimate uses of build scripts such as native library linking to C libraries, code generation such as FFI bindings, protocol buffer compilation and embedding resources, compile-time configuration such as checking for optional dependencies, setting flags based on the environment and current platform, calling external tools and more.

However, the same power that enables build scripts to be a great tool for the previously mentioned use cases, also makes it a very dangerous attack vector, enabling multiple security risks such as:

Arbitrary code execution; the build script has access to the filesystem, network, environment variables, process execution and more.

Attack surface expansion, due to the dependency trees that packages have, the attack surface doesn't consist of just the compiled package or it's direct dependencies but also the entire entire dependency tree.

CI/CD environment targeting: Build scripts are powerful enough to be able to properly detect CI/CD environments using advanced detection that can consist of detecting environment variables, system privileges and information such as the CPU, OS, storage and installed software, existing or missing files and others.

Malicious `build.rs` code can be obfuscated and try advanced detection evasion such as conditional execution, and executing other payloads at runtime if specific conditions are met.

The Rust community has discussed build.rs security multiple times, especially after the previously supply chain attacks.

Sandboxing was one of the proposed solutions, by restricting network and filesystem access, requiring explicit permissions and implementing user prompts.

However as of the time of writing this thesis in 2026, there is no sandboxing present, in part due to the complexity of such a system and the backwards compatibility concerns.

A recommendation is to minimize build.rs pollution, by avoiding using it when it's not necessary, and proper review of changes to build.rs scripts.

Given the lack of sandboxing or other mitigations, heuristic detection can be considered a practical approach to identify malicious build scripts, we can look for patterns such as:

Network library detection, since most build script's shouldn't have the need of making network requests, the presence of HTTP libraries is unusual and be considered a "red flag" in cases where the build script shouldn't be making network connections.

Process spawning is commonly used for invoking tools, however some patterns can be suspicious such as spawning shells such as cmd, powershell, bash and others, executing downloaded binaries or decoding and executing obfuscated code from remote or embedded sources.

Hardcoded raw IP addresses in build scripts is unusual, and legitimate uses usually involve trusted domains and hostnames. The usage of raw IP addresses can suggest the usage of C2 servers or downloading malicious payloads.

Suspicious domain names, such as free or commonly abused TLDs are also suspicious, some TLDs have less strict registration requirements and enforcement.

High entropy strings are very suspicious and can indicate obfuscated and encrypted code or payloads.

Link directive analysis can be also used, since linking to libraries downloaded by the crate from a suspicious source can indicate malicious behavior.

2.6 Large Language Models for Code Analysis

2.6.1 Evolution of Code-Aware Language Models

The application of LLMs to code understanding has rapidly evolved in the past years, starting from models such as CodeBert, Starcoder To Qwen2.5-Coder and Claude Opus.

2.6.2 LLMs for Vulnerability Detection

Several researchers have evaluated LLMs on security tasks such as vulnerability detection.

The Li et al. (2023) "Assessing the Promise and Pitfalls of ChatGPT for Automated Vulnerability Detection" study has evaluated ChatGPT's older 3.5 and 4 models on vulnerability detection, finding that they have a reasonable accuracy on well known patterns, but that they struggle with uncommon vulnerability types and more complex code, and they have a high false positive rate.

”How Well Do Large Language Models Serve as End-to-End Secure Code Producers?” by Zhang et al. (2024) is a evaluation of multiple LLMs on SecurityEval, which found that SOTA LLMs still introduce vulnerabilities in generated code, and that they detect vulnerabilities far better than avoiding generating them at all.

2.6.3 Challenges in Large-Scale LLM Security Analysis

Using LLMs at a large scale such as ecosystem-wide level introduces multiple challenges.

Firstly, the context window is limited even for large context models (such as the 128K+ tokens models), which cannot process larger codebases in a single prompt. We can chunk code into batches, however this can lead to loss of context and has a significant accuracy impact and increased time and cost. We can also prioritize analysis of specific files such as build.rs or files imported in build.rs, however this is also not a perfect solution since malicious code can be hidden in other files or imported dynamically. RAG can also be used however it still has the problem of losing context, increased time and cost and less accuracy than fitting the entire codebase in the context window.

We talked about the computational cost a bit, however LLM inference is computationally expensive in all cases, even with batching and local hosting there still is a significant cost and time cost.

LLMs also have the tendency of hallucinating, which can lead to reporting false positive or false negatives, even changing the output format and conclusion based on small changes or different seeds, even with the temperature set to 0.

Prompt engineering can help with accuracy however it isn’t a complete and perfect solution, and there is no real way of finding the best prompt, longer prompts also occupy more of the context window.

Study	Model(s)	Benchmark	Key Result	Key Limitation
Li et al. [LHZQ23] (2023)	GPT-3.5/4	Big-Vul (190K C/C++ funcs)	F1: ~10% (3.5), ~29% (4)	Trails specialized models; struggles with uncommon vulnerability types
Gong et al. [GCL ⁺ 24] (2024)	GPT-3.5/4, Claude, CodeLlama	SecurityEval (121 tasks, 69 CWEs)	Vulnerable code in > 75% of outputs	Better at detection than secure generation

Table 2.3: Limitations of LLMs in vulnerability detection and secure code generation.

Given these limitations, we decided to leave the LLM module optional, using a conservative threshold and intending it to be used as a tool alongside human review rather than automatically flagging packages as malware or benign.

2.6.4 Local vs. API-Based LLM Deployment

For an ecosystem scale analysis, deployment architecture is key.

API-Based deployment is easier to set up and has no maintenance cost, however it has a cost due to the sheer scale.

Local deployment has the advantages of being cheaper for the inference, however it has a maintenance and investment cost for the hardware.

For this research, we are using a small local deployment of Qwen2.5-Coder-14B, providing privacy and 0 cost per query besides the already owned hardware and electricity cost.

2.7 Secret Detection in Source Code

2.7.1 The Problem of Hardcoded Secrets

Hardcoded secrets represent an ongoing security problem especially in novel packages and software. Secrets such as API keys for cloud providers and services, database connections, private ssh keys, tokens, jwt secrets and others can be accidentally included in commits to the source code.

Developers move fast, sometimes hardcoding during development and forgetting to remove files and committing them to version control.

2.7.2 Secret Detection Approaches

Regular expressions can be treated as a form of signature based detection, using patterns for known secret formats such as Github tokens, private keys, and cloud provider API keys can be an effective way of detecting secrets.

Like encrypted and obfuscated strings, secrets usually have high entropy compared to normal code.

Machine learning approaches can also be a useful tool for secret detection, by training models on known past codebases with revoked secrets can be effective.

2.7.3 Gitleaks

Gitleaks is the industry standard for open source secret detection in git repositories, it uses a rule based approach such as regular expression patterns, entropy thresholds, path patterns, keywords and others. There are over 150 rules present. Gitleaks provides full repository scan, being able to scan all the commits and it can also provide CI/CD integration. It is also a great tool to implement due to it's JSON output, it is easy to implement into existing frameworks, pipelines and infrastructure. Written in go, it is highly performant even on large repositories.

2.7.4 Secrets in the Rust Ecosystem

Rust's crates have permanent publication, they cannot be completely removed, only yanked, which remain downloadable if their version is set before the point of yanking. Developers may accidentally include configurations files, environment files, debug code with secrets and more.

2.8 Existing Tools and Related Work

2.8.1 Rust-Specific Security Tools

The Rust ecosystem has several overlapping but distinct tools. The table below compares them across scope, automation level, model and limitations.

Tool	Primary Scope	Automation in CI/CD	Ecosystem Scale & Sharing	Key Limitations
<code>cargo audit</code>	Known vulnerabilities (RustSec DB)	High - CLI, JSON output, non-zero exit codes	Any crate; no notion of organizational trust	Only known CVEs; no malware or behavior analysis
<code>cargo deny</code>	License, advisory, and dependency bans	High - <code>deny.toml</code> policy checks	Project-local configuration; no community-wide sharing	Requires up-front policy design; not aimed at malware
<code>cargo crev</code>	Distributed code-review trust web	Partial - reviews are manual; verification can run in CI	Scales socially via Web of Trust (e.g., <code>web.crev.dev</code>)	Review effort does not scale well to large ecosystems; niche adoption
<code>cargo vet</code>	Organization-centric audit registry	Designed for CI enforcement (e.g., <code>mozilla-central</code>)	Scales via sharing audit records across organizations	Focused on “safe to run/deploy,” not detecting unknown malware

Table 2.4: Comparison of Rust ecosystem security and auditing tools.

Our framework focuses on automated, ecosystem-wide analysis, rather than per-project analysis.

2.8.2 General-Purpose Security Tools

Gitleaks as described in section 2.7.3, is the standard for open source secret detection in git repositories.

Semgrep is a fast open source static analysis tool, supporting many languages including Rust, having a very large rule library, support of custom rules and pattern matching with semantic awareness.

Dependabot has automated dependency updates and can create pull requests for vulnerable dependencies, and can be integrated with Github and GitLab, however it is not a malware detection tool.

2.8.3 Commercial and Community Supply Chain Security Platforms

Socket.dev provides supply chain attack detection of npm and PyPi packages, metadata anomaly detection and network call analysis, however it lacks Rust support as of now.

Phylum provides supply chain risk assessment with experimental Rust support, machine learning risk scoring, author reputation analysis and dependency relationship analysis.

Besides commercial platforms, open-source initiatives such as Sigstore [Lin24a] also exist, under the Linux Foundation it provides keyless code signing for artifacts. Cosign is one of its components, for signing and verifying artifacts. Rekor is a public log with tamper resistance and are already used by some packages, sigstore-rs is an implementation that allows Rust packages to be signed.

2.8.4 Academic Research on Ecosystem Security

”Understanding Security Threats in Open-Source Software Packages” (Survey papers) provides surveys of attack techniques and defenses, analysis of detection effectiveness and categorization of attacks

"On the Prevalence of Vulnerable Dependencies in the npm Ecosystem" analyzed large scale data on npm.

"A Measurement Study of Security Threats Collected from npm and PyPI" analyzed both npm and PyPI ecosystems, including typosquatting and dependency confusion.

2.9 Distributed Systems for Large-Scale Analysis

2.9.1 Challenges of Ecosystem-Scale Analysis

Analyzing an entire package ecosystem has multiple, highly-complex challenges including:

The scale of the ecosystem, crates contains over 150.000 crates with each having multiple versions, easily spanning over a million releases, analysis would involve downloading each repository, running multiple advanced modules, storing results and managing temporary storage for execution speed.

Dependency ordering, transitive risk propagation is a key feature and it requires analyzing dependencies before the dependents. This heavily complicates parallelization.

Analysis time is very variable, some crates taking seconds to analyze others can take minutes for large codebases, spanning into hours when scanning the entire commit history with LLMs.

Fault tolerance is also a challenge, the ecosystem has multiple broken repositories, failures such as network issues, timeouts and others are common and we must recover gracefully without losing progress.

Resource management is also required, concurrent analysis require disk space management due to the usage of a ramdisk, memory usage is also high together with network bandwidth, cpu usage and database connections.

2.9.2 Work Queue Architectures

A distributed work queue architecture is a common solution for analysis similar to this, we have multiple options for queues.

Redis provides primitives for implementing work queues, 'BRPOPLPUSH' is a command for atomic claim and move operations, it also provides features such as hashes for storing metadata.

RabbitMQ is a message broker providing persistent work queues, message acknowledgement, retry possibilities, load balancing and more.

Apache Kafka provides high throughput, partitioning and more advanced features, however it is more complex to set up and can be overkill for this use case.

For this research, we picked Redis due to familiarity, simplicity performance and native atomic operations which let us implement a simple work queue.

2.10 Summary and Research Gap

The reviewed papers show that even though supply chain security has started to receive more attention from researchers, there are still significant gaps, sup-

ply chain attack taxonomies are established[OPSM20, LPMB23] but they have not been deployed at an ecosystem-wide scale, papers classify attacks without providing pipelines for detection or mitigation, and tools are focused on individual packages and projects instead of the entire ecosystem:

2.10.1 Ecosystem-Specific Gaps for Rust

The rust ecosystem has received far less attention from researchers compared to ecosystems like npm or PyPI despite of its growth in importance, especially in critical infrastructure and fields. No existing work combines vulnerability scanning, build script analysis with secret detection and LLM analysis into a unified comprehensive intuitive solution. Especially, due to build.rs scripts being an extremely powerful feature it is needed to have more research on how to protect upstream dependents.

2.10.2 Methodological Gaps

LLMs have been evaluated for vulnerability detection, however there is a lack of research on deployment at scale on an ecosystem level. Existing tools such as cargo audit work on individual projects, not on the entire ecosystem to provide transitive risk propagation and other more advanced features such as cross-dimensional correlation. Security tools are used in a reactive way, isolated, there is a benefit to correlate findings across multiple "dimensions" (vulnerabilities, secrets, build.rs behavior). Which might be useful for prioritization and revealing more complex patterns.

2.10.3 Contributions of This Thesis

The thesis addresses these gaps by designing a complex, comprehensive optimized framework for analysis integrating multiple modules with a focus on the Rust crates ecosystem, by implementing a distributed architecture, with an in memory work queue, web dashboard and a database that can be used for further analysis. The framework uses multiple modules such as heuristic build script analysis together with existing tools, we are conducting a large scale empirical assessment to more easily identify possible malicious packages and signal possible vulnerabilities. The plan is to open source all of the code, with the possibility of having the Rust foundation integrating parts of it or a modified version into their infrastructure, to provide better security for the entire ecosystem.

Chapter 3

Methodology

3.1 Research Approach

This thesis implements a novel framework with empirical evaluation of its application to the Rust crate ecosystem. The research process has 4 steps.

1. Problem identification and requirement analysis: literature review of supply chain security and attacks, identification of research gap and objectives establishment.
2. Design and development: Iterative implementation and design of the presented framework, keeping in mind the requirement of scalability.
3. Empirical application: Execution against a subset of the crates.io database.
4. Evaluation and discussion: Result analysis, measurements, comparison and future work ideas.

3.2 Threat Model

3.2.1 System Model

The ecosystem has the following principal entities:

- Crate maintainers: Individuals/organizations that develop the crates. They control the repository, source code and metadata.
- Consumers: Developers who have at least a crate dependent on a project.
- Crates.io registry: The centralized repository that stores the crate versions, metadata and sometimes code, and provides them to the consumers.
- Build infrastructure: Cargo, rustc and CI/CD systems.
- End users: Users of software build using crates from crates.io, who can also be affected by any compromise in the supply chain

3.2.2 Attacker Model

We consider attackers with the following skills: Package registration - an attacker can register any crate name allowed by crates.io and with no existing crate Code publication - an attacker can publish crate versions containing arbitrary code Build-time execution - Code can be executed at compile-time using build.rs Metadata manipulation - Keywords, descriptions and documentations from a crate page are written by the authors Ecosystem knowledge - knowledge of popular crate names,

popular keywords, types of crates, naming conventions, etc. Resources - limited resources compared to nation sponsored actors

We assume attackers cannot modify crates infrastructure or compromise it's server, or find any exploits that would allow them to modify crates owned by other maintainers without having direct access to their accounts, or do Man in the Middle (MITM) attacks between victims and crates.io.

3.2.3 Attacker Classes

Class	Motivation	Resources	Examples
"Skid"	Experimentation	Low	Copy-paste/Vibecoded malware
Criminal	Financial gain	Medium	Professionally developed Stealers, C2, ransomware
Advanced Criminal	Targeted financial gain	High	Sophisticated attacks, targeted espionage
Nation-State	Espionage, sabotage	Very High	Long-term infiltration (i.e. xz-utils), infrastructure compromise

The framework primarily targets "skids" and "criminal" attackers, whose techniques are less advanced and more easily detectable using static analysis. Advanced attackers could evade automated detection due to factors like complex obfuscation and understanding of the system.

3.2.4 Attack Vectors Addressed

Attack Vector	Detection Module	Detection Approach
Known vulnerabilities	Cargo Audit	RustSec database matching
Hardcoded secrets	Gitleaks	Regex + entropy analysis
Malicious build.rs	Build Script Heuristics	Network, IP, TLD, process, entropy patterns
Typosquatting	Typosquat Scorer	Multi-algorithm string similarity
Obfuscated/novel malware	LLM Analysis	Semantic code understanding
Transitive compromise	Dependency Propagation	Graph-based risk propagation

3.2.5 Scope and Assumptions

We operate the framework under these assumptions:

1. The source code is available and hosted on a git repository that doesn't rate limit or restrict our access, grabbed from the crates.io database.
2. LLM reliability, due to models becoming better at code reviews, and the framework being easy to extend for API usage with state of the art models
3. Attackers reuse patterns and techniques that can be found through the use of heuristics.

The following are out of scope for this thesis:

- Dynamic/Runtime analysis using sandboxed or emulated execution
- Analysis of compiled binaries
- Analysis of network behavior at runtime (included in runtime analysis)
- Compromised crates through hacked accounts or infrastructure exploits

- Detection of social engineering

3.3 Data Collection Strategy

The crate metadata source is the crates.io database dump published by the crates.io team, containing CSV files of all published crates including metadata, such as name, description, repository url, etc; version information, download statistics, and dependency relationships

These CSV files are processed and inserted into a PostgreSQL database using a custom pipeline that resolves dependency references and constructs a dependency graph. The source code of the repositories is obtained by cloning the last commit of the git repository from the crate metadata.

The system supports a configurable scope using the 'use_only_first_x_crates' field from the configuration toml file, together with the single crate analysis parameter, allowing the analysis of a single crate and its dependencies.

3.4 Analysis Framework Design

The design was focused on the following principles:

- Modularity: Having independent modules that can removed or added easily, possibly requiring DB changes
- Horizontal scalability: We have the possibility of adding worker nodes from a single or multiple machines to increase throughput
- Topological processing: Processing dependencies before dependents to enable transitive risk propagation
- Fault tolerance

3.5 Evaluation Metrics

The evaluation will consist of two parts:

Security analysis

- Number and severity distribution of vulnerabilities
- Number, types, average/median entropy of secrets
- Characteristics of build scripts
- Distribution of LLM scores
- Number and quality of typosquatting candidates
- Transitive risk propagation statistics

System performance

- System Throughput
- Worker speed-up scale
- Resource utilization

Chapter 4

System Architecture

4.1 High-Level Architecture Overview

The framework uses a producer-consumer architecture with a central layer, the main components are:

1. Data Pipeline - Imports all the crate information required from the crates.io database into PostgreSQL
2. Producer - Constructs the Redis dependency graph and inserts crates with no unprocessed dependencies into the queue
3. Worker - It claims a crate from the Redis ready queue, runs all modules and inserts the results into PostgreSQL
4. Final processing: Computes transitive dependency risk after all analyses are finished.
5. Web Panel - A browser interface for visualizing results

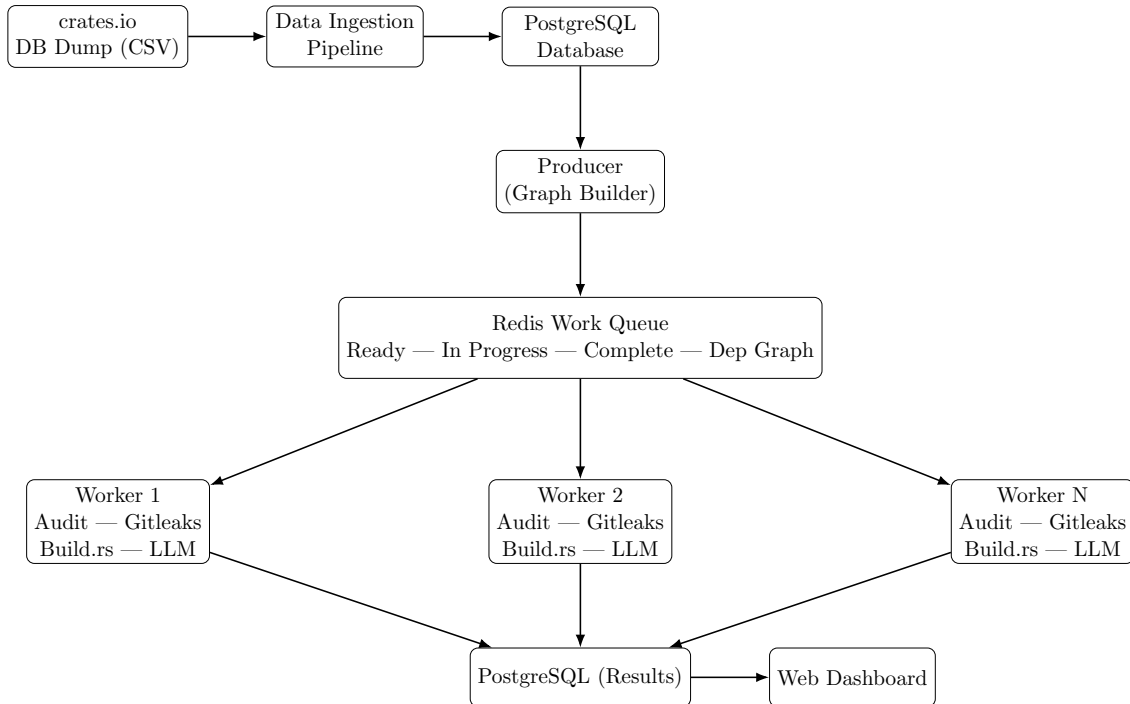


Figure 4.1: High-level architecture of the distributed crate analysis framework.

4.2 Data Ingestion Pipeline

This pipeline processes CSV files from the crates.io database dump archive:

- crates.csv - contains core metadata (ID, name, repository URL, etc.) - crate_downloads.csv - download count for each crate - versions.csv - version specific metadata - dependencies.csv - crate dependency associations between versions, including the dependency type and version id

These CSVs are all processed, firstly we resolve crate-version relationships and after processing we perform batch inserts into PostgreSQL.

4.3 Dependency Graph Construction

The dependency graph is a DAG where nodes represent crates and edges represent dependency relationships. The graph is build by:

1. Query PostgreSQL for all dependencies
2. For each crate build a dependency set in Redis, tracking unprocessed dependencies.
3. Build a reverse index to be able to get which crates depend on a target crate.
4. Pushing crates ready to process (with zero unprocessed dependencies) into the Redis ready queue.

After a worker analyses a crate, it has to atomically update the dependency sets of all that crate's dependent crates. If any of these's sets becomes an empty set, it is pushed into the ready queue.

Dependency cycles are detected using depth first search and are resolved by breaking cycle edges.

4.4 Distributed Work Queue

The Redis queue has the following structures:

Redis Key	Type	Purpose
crate:ready	List	FIFO queue of crate IDs ready for analysis
crate:in_progress	Set	Crate IDs currently being analyzed (atomic claim)
crate:completed	Set	Crate IDs successfully analyzed
crate:failed	Set	Crate IDs that failed analysis

A worker uses 'BRPOPLPUSH' to do an atomic claim of a crate, this ensures there are no race conditions. Failed crates are marked as failed but are not re-queued and do not block dependent crates from being processed. This can be further improved if

4.5 Database Schema Design

The PostgreSQL database contains the following tables:

Table	Description	Key Fields
<code>crates</code>	Crate metadata from crates.io	id, name, repository, crate_downloads, timestamps
<code>dependencies</code>	Dependency edges between crates	crate_id, dependency_id (composite PK)
<code>scan_results</code>	Aggregated analysis results per crate	FK to crates, LLM score, boolean flags, build.rs heuristics, entropy
<code>cargo_audit_results</code>	Individual vulnerability findings	UUID PK, FK to crate, rustsec_id, CVSS score
<code>gitleaks_results</code>	Individual secret findings	UUID PK, FK to crate, rule_id, secret, location, entropy
<code>typosquat_results</code>	Potential typosquatting pairs	FK pair to crates, multiple similarity scores + combined score
<code>runner_metadata</code>	Analysis run tracking	timestamp PK, last checked crate

The schema was designed to use indexing for commonly used queries for filtering, combined scores and crate ids.

4.6 Web Dashboard

The web dashboard is implemented as a standalone project using Axum and provides an API for querying the database. The frontend supports the following features:

- Dashboard view - Showing high level statistics such as crates analyzed, vulnerability count, secrets found and others providing charts for easy visualization.
- Crates browser - a paginated, sortable and filterable list of all the analyzed crates.
- Scan results view - A filterable, sortable and paginated list of all crates with their metadata.
- Scan results page - Same as the others in terms of feature, showing information such as build.rs heuristics, vulnerability results and more.
- Vulnerability viewer - Lists all cargo audit findings for all crates with the severity score and links to the advisory page.

- Secrets viewer - Shows all gitleaks findings for all crates, with rule IDs, entropy and location in the repository.
 - Typosquatting viewer - Shows all typosquatting pairs with all of the similarity scores from all algorithms used, together with the combined score
 - Dependency browser - Shows the dependencies between crates.
- All endpoints support pagination, sorting and multiple filtering.

4.7 Deployment and Infrastructure

The workers are containerized using Docker to ensure easy scalability on a single or multiple machines, the sample production deployment is the following:

- PostgreSQL database with persistent storage
- Redis for the work queue and dependency graph
- Worker containers, by using a custom Dockerfile we can scale multiple workers using 'docker-compose up -d --scale worker=N' where N is the number of workers to start. Each worker has a limit of 2 cores and 4GB of RAM.

To achieve maximum I/O performance the analysis is done on a temporary mounted RAM disk at /tmp/crates, eliminating the disk I/O bottleneck during cloning and analysis.

Multiple machines can be used by configuring the workers on multiple machines to connect to the central Redis instance, this enables scaling across a cluster if needed.

Chapter 5

Analysis Modules

5.1 Cargo Audit - Vulnerability Scanning

The cargo audit module uses cargo audit, which is integrated with the RustSec advisory database, a maintained database of security advisories for crates. For each crate's local git clone of the latest hash the module executes the command 'cargo audit -json -no-fetch', the output a JSON from which we can extract vulnerability information and severity scores.

Severity scores are calculated by implementing the CVSS v3.1 calculation from the raw vector strings. Scores are normalized to a 0-100 scale for comparison.

Cargo audit cannot run locally in parallel due to a shared lock, to mitigate this we used docker containers for parallelization.

Results are stored in the 'cargo_audit_results' table and 'scan_results' table for aggregation.

5.2 Gitleaks - Secret Detection

The Gitleaks module detects hardcoded secrets, API keys, tokens, and other credentials present in the source code. Gitleaks output JSON is parsed to extract the following information:

- Rule ID - which represents the type of detected secret
- Secret - The detected value
- Location - Path and line number of the secret from the repository
- Entropy - Calculated shannon entropy by gitleaks of the string, which can indicate if it's a real secret or a false positive

The results are stored in the 'gitleaks_results'. The module is prone to returning false positives due to test secrets present in most of the cases.

5.3 Build Script Analysis

The build script analysis module has six heuristic implementations to detect potential malicious behavior:

- Network call detection - Strings related to network libraries and calls such as 'reqwest', 'hyper', 'curl', 'ureq', 'tokio::net', 'std::net::TcpStream' and others. Net-

work calls, as previously mentioned, can be used to download payloads or exfiltrate data.

- Link directive detection - Using Regex, we look for patterns such as link attributes, mostly used for FFI and non malicious behaviors, these can be used to link to malicious libraries.

- Shannon entropy calculation - High values can indicate obfuscated or encoded data and code, we also compute an average entropy of all '.rs' files from the repository to compare against.

- Process spawning detection - Using Regex, we look for patterns such as 'Command::new()', '.exec()', '.spawn()' and others. Commonly used to invoke external tools, however it can also be used to execute payloads, attackers often pair this with network calls to download the malicious payload.

- Raw IP addresses - Regex based detection of IP addresses, we exclude local and private ranges, however non private and non local IPs usually are an indicator of suspicious behavior instead of using domain names.

- Suspicious TLDs - Using Regex, we look for free and commonly used TLDs by attackers, such as '.tk', '.ga', '.cf' and others. These TLDs are often used due to them being free or having a low cost of rental.

5.4 LLM-Assisted Malicious Code Detection

The LLM analysis module uses a locally hosted LLM using LM Studio to perform analysis of crate source code. This module is optional due to extremely high computational cost compared to the rest of the modules. It operates in the following way:

- Collects all source code files from a target extension list
 - Source files are chunked to batches that fit into the model context window. Files are prioritized by relevance such as 'build.rs', 'lib.rs', 'main.rs' and 'mod.rs'
 - Each batch is sent to the LLM together with a prompt that instructs the LLM to evaluate the code for potential malicious behavior, such as reverse shells, unauthorized network access, data exfiltration, cryptocurrency mining and more.
 - The LLM returns a score from 0 to 100 together with a summary for each batch. The final score is set to the maximum score across the batches.
 - Crates with a score above the set threshold are flagged for human review.
- The module has a five minute timeout per batch.

5.5 Typosquatting Detection

The typosquatting detection module computes pairwise name similarity between crates in parallel using multiple string similarity algorithms:

Name	Weight	Notes
Levenshtein Distance	20%	Minimum number of single-character insertions, deletions, and substitutions required to transform one string into another.
Damerau-Levenshtein Distance	25%	Levenshtein with consideration of character transpositions
Jaro-Winkler Similarity	25%	Computes character-level matching with a bonus for common prefixes.
Keyboard Proximity Distance	15%	Based on the physical distance between characters on a QWERTY keyboard layout, using Euclidean distance between key positions.
Prefix Similarity	15%	Measures the length of the common prefix relative to the string lengths, with a bonus for prefixes of 3 or more characters.

The combined score, after applying the weights, is normalized to a 0-100 scale.

5.6 Transitive Dependency Risk Propagation

After the complete analysis, another step computes the 'has_malicious_dependencies' flag for each crate. The computation is done via a single SQL UPDATE query.

Chapter 6

Implementation Details

6.1 Technology Stack

Component	Technology	Justification
Core application	Rust	Memory safety, performance, async and concurrent support
CLI framework	Rust + Clap	Easy command definition with subcommands
Async runtime	Tokio	Industry-standard Rust async runtime, speed and ease of use
Database ORM	Diesel (async)	Type-safe SQL queries, fastest orm, migration management
Connection pooling	bb8	Fastest async connection pool for PostgreSQL
HTTP client	reqwest	Easy async HTTP for LLM API communication
JSON parsing	sonic_rs	High-performance JSON parser (faster than serde_json)
Work queue	Redis	Minimal latency, atomic operations, Lua scripting
Data storage	PostgreSQL	Fast, Relational integrity, complex query support, JSONB
Web framework	Axum	Async, Ease of use, performance, Tower middleware ecosystem
Parallelism	Rayon	Easy parallelism for typosquatting analysis
Logging	tracing + tracing-subscriber	Structured logging with file and console output, helpful for debugging and information
Containerization	Docker + Docker Compose	Reproducible and easy deployment, horizontal scaling

6.2 CLI Application Structure

The CLI app has the following CLI commands and structure:

Command	Description
update-database	Downloads and imports the crates.io CSV dump into PostgreSQL
run-analysis	Constructs dependency graph and populates Redis queue
worker	Starts a worker process that consumes from the queue, used from docker
compute-dependencies	Post-processing for dependency risk flags
queue-status	Displays Redis queue statistics
recover-stale	Recovers items from crashed workers
clear-queue	Resets the entire queue state
fix-cycles	Detects and resolves dependency graph cycles
scan-crate --name X	Single-crate analysis with full dependency tree
queue-crate --name X	Queues a single crate and its transitive dependencies
check-llm	Verifies LM Studio connectivity
typosquat	Runs typosquatting detection on the entire database

6.3 Redis Queue Implementation

The redis queue implementation was designed to handle multiple complex scenarios:

- Topological ordering of crates to ensure the dependents are analyzed first
- Atomic operations - Atomic claim of crates, handling crate completion, dependent updates and others happen in a atomic manner to prevent race conditions
- Blocking pop - BRPOP is used to avoid busy waiting
- Statistic tracking - The queue has counters for total crates, processed crates and others to provide monitoring capabilities.

6.4 Cycle Detection and Resolution

Dependency cycles can prevent crates from ever being processed as dependency cycles will cause crates to always have dependencies.

- Cycle detection - Using depth first search, we identify cycle edges
- Cycle breaking - Removal of edges allows crates to be processed, 'fix-cycles' is used for simple A-B-A cycles, and 'break-cycles' is used for more complex cycles.

6.5 Worker Lifecycle and Fault Tolerance

Each worker is an independent instance with a lifecycle that does;

- Initialization - Connects to PostgreSQL and Redis and sets up the state and handlers
- Main loop - Claim a crate from the ready queue in an atomic way, download its depository and run each module, after that insert the results into PostgreSQL and remove the crate from Redis.

- Shutdown - handle signals like SIGINT and SIGTERM by finishing the current crate and only after that exiting.

- Failed analysis produce placeholder results.

6.6 Batch Processing and Database Optimization

- Batch updating - INSERT and on conflict UPDATE statements for results
 - Chunked imports - The database population from the CSV files is done in chunks to improve performance and reduce memory usage
 - Connection pooling - bb8 is used to manage database connections
 - Indexing - Indexes on commonly used queries in the codebase are used, they do not increase the database size enough to be a problem

Chapter 7

Results and Evaluation

7.1 Dataset Overview

Table 7.1: Crate scanning statistics.

Status	Count	% of total
Total crates in database	264,750	100%
Crates with scan records	260,912	98.6%
Successfully scanned ($\neq -1$)	226,546	85.6%
Failed scans ($= -1$)	34,366	13.0%
Not attempted	3,838	1.4%

On this run, 264.750 crates were present in the crates.io database dump. A small percentage of crates have private repositories or have blocked access on some IPs, and 13.0% failed to be scanned due to reasons including a missing Cargo.lock file, making the analysis with cargo audit not possible, a percentage of 85.6% of crates were successfully scanned. The percentage of vulnerable crates will use the 226.546 number of the successfully scanned crates to avoid skewing results related to crates with unknown status.

Out of the 226.546 successfully scanned crates, 72.773 (32.1%) had at least one vulnerability, which would be 1 in 3 crates.

Table 7.2: Summary statistics for scanned crates.

Metric	Value
Total crates in database	264,750
Successfully scanned	226,546
Failed / never attempted	38,204 (14.4%)
Crates with ≥ 1 vulnerability	72,773 (32.1% of scanned)
Max vulns in a single crate	83
Mean vulns per crate (scanned)	~ 2.13
Median vulns per crate (scanned)	0
Mean dependencies per crate	6.47
Median dependencies per crate	4

The median crate has zero vulnerabilities, but the mean is 2.13 due to some unmaintained crates that have a large number of vulnerabilities. Simlh, the "most

vulnerable” crate, has been last updated over 5 years ago, the same case is for `holochain_core_types`, `in_stream`, `holochain_locksmith`, `oatie`, which are the 5 most vulnerable crates by cargo audit vulnerability count. Due to a minority of the crates having a large number of vulnerabilities the mean and median are quite different.

Vulns per crate	Count	% of scanned
0	~153,773	~67.9%
1	~19,054	~8.4%
2	~8,412	~3.7%
3–4	~12,891	~5.7%
5–9	~16,496	~7.3%
10+	~15,920	~7.0%

Table 7.3: Distribution of vulnerabilities per scanned crate

Approximately 68% of scanned crates have zero known vulnerabilities, however 7% have at least 10 vulnerabilities, we can see that there is a significant count of unmaintained crates.

7.2 Vulnerability Landscape

Table 7.4: Vulnerability severity distribution (CVSS 3.1).

Severity	Findings	%	Affected crates
Critical (9.0–10.0)	8,841	1.8%	4,917
High (7.0–8.9)	39,536	8.2%	18,258
Medium (4.0–6.9)	30,717	6.4%	23,177
Low (1.0–3.9)	5,148	1.1%	2,461
Info (< 1.0)	398,783	82.6%	67,785
Total	483,025	100%	—

The majority of findings (82.6%) are informational only, composed of mostly advisories for unmaintained or deprecated crates. A small percentage of 11% of findings are high or critical.

Table 7.5: Most vulnerable crates by weighted risk score.

Crate	Vuln Count	Risk Score	Notes
<code>sim1h</code>	83	highest	>5 yrs unmaintained
<code>holochain_core_types</code>	62	very high	unmaintained
<code>in_stream</code>	58	high	unmaintained
<code>holochain_locksmith</code>	58	high	unmaintained
<code>oatie</code>	52	high	unmaintained
<code>sp-runtime</code>	41	medium	567 dependents
<code>sp-core</code>	41	medium	473 dependents
<code>sqlx</code>	12	low	widely used ORM, 2000+ deps
<code>tokio-rustls</code>	9	low	TLS integration, 1000+ deps

The most vulnerable crates are unmaintained, confirming abandonment instead of malicious intent, this is the primary vulnerability count factor in the ecosystem.

7.3 Dependency network

Dependencies	Count	%
0	~43,729	~16.5%
1–4	~96,868	~36.6%
5–9	~64,836	~24.5%
10–19	~43,762	~16.5%
20–49	~14,597	~5.5%
50–99	907	0.3%
100–199	47	0.02%
200+	4	0.002%

Table 7.6: Distribution of dependency counts across crates

Most crates have a small number of dependency, however the ecosystem has over 1.7 million dependency relationships.

Crate	Dependents	Notes
<code>serde</code>	89,672	–
<code>serde_json</code>	63,881	–
<code>thiserror</code>	49,325	–
<code>tokio</code>	45,351	–
<code>clap</code>	36,323	Has 6 vulns

Table 7.7: Top 5 most depended-upon crates

The most depended upon crates are mostly vulnerability free as of the current scan, showing that the core crates are well-maintained, however `clap` according to cargo audit has 6 known vulnerabilities which could affect over 36.000 of downstream crates.

Some crates are both highly depended upon and also have known vulnerabilities, some of these are:

Crate	Dependents	Vulnerabilities
<code>sp-runtime</code>	567	41
<code>sp-core</code>	473	41
<code>sqlx</code>	2,411	12
<code>tokio-rustls</code>	1,434	9
<code>prost</code>	3,699	2

Table 7.8: Popular crates with notable vulnerability counts

`sp-runtime` has `sp-core` as a dependency which makes it also appear as highly vulnerable.

A single advisory (RUSTSEC-2023-0071) propagates to multiple crates due to the dependency on `sp-core` or crates that depend on `sp-core`.

Excluding entries that have 'test' or 'example' in the file location, there have been 882.955 secrets found across 18.571 crates. However almost all of the found secrets are false positives of the gitleaks rule 'generic-api-key', besides this, even for the rest of the rules almost all detections are still false positives. Even some other rules like `openai-api-key` after being manually checked, contain false positives due to keys mimicking the patterns but not being actual valid keys, in this exact case they are being comments for examples.

Type	Occurrences	%	Crates
generic-api-key	845,625	95.8%	16,387
private-key	18,168	2.1%	3,646
curl-auth-header	9,631	1.1%	1,849
jwt	1,885	0.2%	917
github-pat	1,512	0.2%	561
gcp-api-key	1,136	0.1%	412
aws-access-token	1,094	0.1%	438
stripe-access-token	923	0.1%	511
openai-api-key	2	< 0.01%	1
All others	3,983	0.5%	—

Table 7.9: Distribution of detected secret types

The mean entropy is 3.87, median entropy is 3.81, p95 is 4.83 and max entropy is 5.91. Higher entropies indicate potential secrets. However due to the high amount of false positives entropy is not a good way to filter results, however it can filter out low entropy "fake" secrets.

Severity	Findings	%	Affected crates
Critical (9.0–10.0)	8,841	1.8%	4,917
High (7.0–8.9)	39,536	8.2%	18,258
Medium (4.0–6.9)	30,717	6.4%	23,177
Low (1.0–3.9)	5,148	1.1%	2,461
Info (< 1.0)	398,783	82.6%	67,785

Table 7.10: Distribution of vulnerability severities

Most vulnerabilities are informational, most being from unmaintained and deprecated crates. Even though most vulnerabilities are not exploitable, there is a supply-chain hygiene problem.

The most common advisory that has a severity score is RUSTSEC-2023-0071, with a 5.9 severity and 12.487 occurrences.

The raw ip analysis found 13 results, which are all false positives, stemming from stuff like setup files version numbers, user agents.

Table 7.11: Prevalence of suspicious `build.rs` patterns.

Pattern	Crates	% of scanned
Process Spawning	3,194	1.41%
Link Directives	2,403	1.06%
Network Calls	1,005	0.44%
Free TLDs	36	0.02%
Raw IP Addresses	13	< 0.01%

7.4 Suspicious `build.rs` Analysis

Process spawning is one of the most powerful attack vectors, most instances are legitimate however from manual checking. Crates combining process spawning with network calls can be considered more suspicious and should be manually verified.

All 13 raw ip address usage instances found are false positives coming from version strings in setup files, user-agent strings and documentation pieces. No crate was found to be making network calls to raw IP addresses in its `build.rs`.

The `typosquat` detection feature is not as useful for `crates.io` compared to other ecosystems due to the "patterns" of naming crates, such as `opentelemetry-appender` and `opentelemetry-appender2`, `safer_ffi-proc_macro` and `safer_ffi-proc-macros`, most high score pairs are versioned variants coming from the same author (`proc-macro-error-attr2` and `proc-macro-error-attr`). `Crates.io` has heavy usage of suffixes and prefixes such as `-2`, `-rs`, `-sys`, etc.

Only 3.5% of the scanned crates include binaries, from these, 67.7% have known vulnerabilities, more than double the overall percentage.

7.5 Temporal Trends

Table 7.12: Crate creation rate versus vulnerability prevalence over time.

Year	Crates Created	% with Vulns
2014	598	16.1%
2016	3,594	25.3%
2018	7,706	24.4%
2020	17,120	28.9%
2022	25,529	23.6%
2024	36,050	26.8%
2026	51,809	31.1%

The ecosystem had consistent growth, with the vulnerability rate remaining stable at 24-30%, with a slight increase in the last 2 years. This is likely due to better RustSec coverage.

The most downloaded crate have the lowest vulnerability rate and lowest average vulnerabilities. High usage crates are more likely to be better maintained. The medium and low velocity crates are having the highest theoretical risk, being popular enough to have users but not enough to have consistent reviews.

The analysis pipeline had cargo audit take most of the time to run.

Table 7.13: Download velocity versus vulnerability rate.

Velocity Bracket	% Vuln	Avg Vulns/Crate
High ($\geq 1\text{K/day}$)	25.2%	1.25
Medium (100–1K/day)	31.3%	2.54
Low (10–100/day)	32.1%	2.29
Very low ($< 10/\text{day}$)	26.2%	1.69

Metric	Value
Median analysis time	1.46 seconds
Mean analysis time	15.1 seconds
p95 analysis time	86.8 seconds
Max analysis time	52.1 minutes
Dominant component	Cargo-audit (avg 11.2s)

Table 7.14: Analysis Runtime Statistics

74% of the mean time is taken by cargo audit, while gitleaks on the second place takes 3.1s on average. The max time analysis is very big compared to mean and median, after manually checking, there are more crates that have similar analysis time and most are due to cargo audit, while some due to the LLM component, cargo audit seems to have problems updating the ‘tuna’ index, even when running the command manually on the max analysis time crate. We can see that most runs are fast, however there is a right skew, possibly due to some crates lacking a repository or a Cargo.lock file.

We used the model “google/gemma-4-e4b” for the LLM analysis and we scanned the top 10 crates by vulnerability count. The biggest score is 95, for ‘holochain-core_types’, due to it containing a file at the path crates/trycp_server/src/main.rs that runs `Command::new("bash").args(&["-c", arbitrary_command])`

Crate oatie has the second biggest score, of 75, however the LLM doesn’t find any highly suspicious code, and neither does manual checking do.

Table 7.15: Distribution of detected secret types (non-test, non-example locations).

Type	Occurrences	%	Crates
generic-api-key	845,625	95.8%	16,387
private-key	18,168	2.1%	3,646
curl-auth-header	9,631	1.1%	1,849
jwt	1,885	0.2%	917
github-pat	1,512	0.2%	561
gcp-api-key	1,136	0.1%	412
aws-access-token	1,094	0.1%	438
stripe-access-token	923	0.1%	511
openai-api-key	2	$< 0.01\%$	1
All others	3,983	0.5%	—

The generic api key is an overwhelming false positive, it matches high entropy strings resembling API keys, including constants and test data. The Gitleaks approach is comprehensive but produces too many false positives for fully automated

Table 7.16: Entropy statistics for detected secrets.

Metric	Value
Mean entropy	3.87 bits
Median entropy	3.81 bits
p95 entropy	4.83 bits
Max entropy	5.91 bits

analysis.

Chapter 8

Discussion

8.1 Comparison with Existing Approaches

SBOM approaches [Lin24b, OWA24] provide transparency but do not detect malicious behavior.

8.2 Limitations

8.2.1 Detection Limitations

False negatives:

- Novel patterns and attacks can evade our heuristics
- Custom encoding might be done to try to evade obfuscation detection
- Social engineering and account overtaking are not addressed

False positives:

- Legitimate crates with unusual behavior that still remain benign
- Test code may trigger alerts
- High entropy with legitimate use cases
- Common naming patterns can produce false alerts of the typosquatting features

8.2.2 LLM-Specific Limitations

- LLMs might hallucinate vulnerabilities or hallucinate that benign code is malicious or vice versa

- The very limited context window is a problem due to some libraries having large codebases, which are chunked on the current implementation, which can make the model miss cross-file malicious behavior (behavior that present in two separate files is benign on its own but either imports or api usage makes it potentially malicious)

- Reproducibility is another limitation since LLM outputs vary even with low or zero temperature parameter.

- Prompt injection is another problem, adversaries might contain specially encoded unicode characters or try to do prompt injection in comments to evade the model

8.2.3 Scalability and Resource Limitations

LLM inference is extremely computationally expensive for an ecosystem deployment.

Analysis time can take a very long time, especially on large crates, adversaries could do a DoW attack by spamming fake crates with large codebases to attack crates.io.

Storage is another problem if storing crates is required, besides us only cloning with a depth of one and only one branch, some repositories are still large

Besides storage, network bandwidth is also a potential problem, especially if the git services used rate limit us.

8.2.4 Ground Truth Limitations

Validation is extremely hard to do, since only a very small number of existing crates have been documented in a public way as malicious

We also have an unknown rate for false negatives since we cannot know how many attacks exists undetected

8.2.5 Generalizability Limitations

The framework is designed for crates.io and Rust, adoption for other ecosystems would require a full rewrite.

Besides the tools like cargo audit being adapted for Rust, also the build.rs heuristics are also language specific

8.3 Threats to Validity

The CVSS scoring and string similarity algorithms have been tested via unit tests, however we can never know if there are any edge cases

Results are limited to crates with valid repository links in the metadata, some crates have been moved from the link from the metadata and moved to docs.rs (i.e. Find_Matt_C)

The LLM score is only a guide and cannot guarantee anything, be it either the presence of malicious behavior or its absence.

Chapter 9

Conclusions and Future Work

9.1 Summary of Contributions

The thesis aims to design, implement and evaluate a security analysis framework for Rust’s ecosystem. The results demonstrate that automated multi-dimensional analysis is effective and feasible, with some limitations including false positives and the need for manual verification.

1. **A modular and scalable analysis framework.** We designed and implemented a producer consumer architecture with a Redis-based work queue that enforces topological ordering. The system supports worker containers, handles crash recovery and graceful shutdown. A full ecosystem scan of 226,546 crates can be completed with consumer hardware.
2. **Five integrated modules.** The framework combines:
 - Vulnerability scanning using `cargo-audit`, with a CVSS scoring implementation
 - Secret detection from `gitleaks` with rule pattern matching and entropy analysis
 - Heuristic `build.rs` analysis covering multiple patterns (network calls, process spawning, raw IPs, link directives, free TLDs, entropy)
 - Multi-algorithm typosquatting detection with weighted combined scoring
 - LLM-based malicious code scoring via LM Studio’s API
3. **The largest empirical security assessment of crates.io to date.** Scanning 226,546 crates (85.6% of the registry) resulted in the following key findings:
 - 32.1% of scanned crates (72,773) have known vulnerabilities; 82.6% of findings are informational severity, while $\sim 11\%$ are High/Critical.
 - 882,955 secrets were detected across 18,571 crates, but 95.8% are `generic-api-key` false positives.
 - 1.41% of scanned crates (3,194) spawn processes in `build.rs`, representing a compile-time attack surface present across thousands of crates.
 - 38,495 name-similarity pairs were detected; 891 with very high similarity (≥ 95). Some have high download asymmetry suggesting possible impersonation.

- 13% of crates (34,366) could not be analyzed, a significant blind spot reflecting private repositories and limitations of the current implementation, missing `Cargo.lock` files, and clone failures.
4. **Evidence-based risk characterization.** Beyond raw counts, the analysis revealed structural patterns with practical implications:
 - Popular crates are safer high download crates have a 25.2% vulnerability rate versus 32.1% for medium download crates.
 - Newest crates are most vulnerable (30.9% rate for crates under 6 months), likely due to unpatched dependencies and less review.
 - Crates shipping executables have a 67.7% vulnerability rate; more than double the average.
 - The Substrate/Polkadot ecosystem is the largest source of transitive vulnerability risk due to a single advisory from the dependency `sp-core`.
 5. **A negative result for LLM-based module.** Deploying a 14B-parameter LLM at ecosystem scale was not possible due to hardware limitations, scanning the most vulnerable crates didn't produce meaningful findings.

9.2 Future Directions

The framework and findings show several opportunities for future work:

1. **Version-level dependency analysis.** The current implementation tracks vulnerabilities at the crate level, without tracking specific versions. The `versions` and `dependencies` tables already contain version-level data, with the pipeline logic being the missing component.
2. **Integration with crates.io infrastructure.** The `build.rs` heuristic flags could trigger automated warnings on the crate's crates.io page, alerting users to compile-time execution before they add the dependency.
3. **Gitleaks rule set specialization.** Gitleaks has a very high false positive rate in this context. Even after removing test and example paths the remaining are too many for manual review.
4. **Caching and incremental scanning.** Caching cargo audit's artifacts such as different compiler versions and implementing incremental analysis that only re-analyzes crates whose source code or dependency graph has changed would reduce the scan time.
5. **Sandboxed build script execution.** A very important gap in the current implementation is the lack of understanding what the `build.rs` script executes at runtime. Implementing a sandbox and monitoring behavior would be a great addition, especially with a classifier to determine possible maliciousness.
6. **Continuous monitoring.** Integrating the framework with crates.io's real-time feed (instead of the current "periodic" database dump) would enable analysis of newly published crates. The redis queue already is capable of handling a continuous addition of crates.

7. **Improved LLM integration.** Deploying a larger model with RAG for cross-file context, would be a great addition, especially deployed on a cluster for fast inference, with the current implementation was not properly tested due to hardware limitations.
8. **Metadata anomaly detection.** Analyzing patterns in maintainer and ownership changes, release frequency, and version-tag rewriting would complement the current analysis methods.

Bibliography

- [GCL⁺24] Yinhao Gong, Bian Chen, Zhenyu Li, et al. Evaluating the Security of Large Language Models in Code Generation: A Study with SecurityEval. In *Proceedings of the 2024 IEEE/ACM International Conference on Software Engineering (ICSE)*, 2024.
- [LHZQ23] Haonan Li, Yu Hao, Yizhuo Zhai, and Zhiyun Qian. Assessing the Promise and Pitfalls of ChatGPT for Automated Vulnerability Detection. *arXiv preprint arXiv:2308.12434*, 2023.
- [Lin24a] Linux Foundation. Sigstore: Software Signing for Everybody, 2024.
- [Lin24b] Linux Foundation. Software Package Data Exchange (SPDX) Specification 2.3, 2024.
- [LPMB23] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. A Taxonomy of Attacks on Open-Source Software Supply Chains. In *Proceedings of the 44th IEEE Symposium on Security and Privacy (S&P)*, 2023.
- [Ope24] OpenSSF. Supply-chain Levels for Software Artifacts (SLSA) v1.0, 2024.
- [OPSM20] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. Backstabber’s Knife Collection: A Review of Open Source Software Supply Chain Attacks. In *Proceedings of the 17th Conference on Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, 2020.
- [OWA24] OWASP Foundation. OWASP CycloneDX Specification 1.6, 2024.
- [Pyt23] Python Software Foundation. Malware Detection on PyPI, 2023.
- [Son26] Sonatype. 2026 State of the Software Supply Chain, 2026.
- [ZSTP19] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. Small World with High Risks: A Study of Security Threats in the npm Ecosystem. In *Proceedings of the 28th USENIX Security Symposium*, 2019.

Appendix A

Database Schema

```
CREATE TABLE crates (  
    id BIGINT PRIMARY KEY,  
    name TEXT NOT NULL,  
    repository TEXT NOT NULL,  
    crate_downloads BIGINT NOT NULL,  
    db_created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    db_updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    crate_created_at TIMESTAMP NOT NULL,  
    crate_updated_at TIMESTAMP NOT NULL  
);
```

```
CREATE TABLE dependencies (  
    crate_id BIGINT NOT NULL REFERENCES crates(id),  
    dependency_id BIGINT NOT NULL REFERENCES crates(id),  
    db_created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    db_updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    PRIMARY KEY (crate_id, dependency_id)  
);
```

```
CREATE TABLE scan_results (  
    id BIGINT PRIMARY KEY REFERENCES crates(id),  
    has_malicious_dependencies BOOLEAN NOT NULL,  
    llm_malicious_score SMALLINT NOT NULL,  
    llm_notes TEXT,  
    has_executable_files BOOLEAN,  
    cargo_audit_max_dep_score SMALLINT,  
    cargo_audit_vulns_count SMALLINT,  
    build_rs_network_calls BOOLEAN,  
    build_rs_has_link_directive BOOLEAN,  
    build_rs_entropy_score DOUBLE PRECISION,  
    build_rs_has_process_spawning BOOLEAN,  
    build_rs_has_raw_ip BOOLEAN,  
    build_rs_has_free_tlds BOOLEAN,  
    entropy_score DOUBLE PRECISION  
);
```

```
CREATE TABLE cargo_audit_results (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    crate_ BIGINT REFERENCES crates(id),  
    rustsec_id VARCHAR(10) NOT NULL,  
    severity SMALLINT  
);
```

```
CREATE TABLE gitleaks_results (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    crate_ BIGINT REFERENCES crates(id),  
    rule_id VARCHAR(255) NOT NULL,  
    secret VARCHAR(255) NOT NULL,  
    loc VARCHAR(255) NOT NULL,  
    entropy DOUBLE PRECISION NOT NULL  
);
```

```
CREATE TABLE typosquat_results (  
    id BIGSERIAL PRIMARY KEY,  
    crate_id BIGINT NOT NULL REFERENCES crates(id),  
    similar_crate_id BIGINT NOT NULL REFERENCES crates(id),  
    levenshtein_score SMALLINT NOT NULL,  
    damerau_levenshtein_score SMALLINT NOT NULL,  
    jaro_winkler_score SMALLINT NOT NULL,  
    keyboard_distance_score SMALLINT NOT NULL,  
    prefix_similarity_score SMALLINT NOT NULL,  
    combined_score SMALLINT NOT NULL,  
    db_created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    UNIQUE (crate_id, similar_crate_id),  
    CHECK (crate_id != similar_crate_id)  
);
```

```
CREATE TABLE runner_metadata (  
    run_time TIMESTAMP PRIMARY KEY,  
    last_checked_crate BIGINT REFERENCES crates(id)  
);
```

Appendix B

Configuration Reference

```
# Database
max_connections = 5
connection_string = "postgres://postgres:postgres@localhost:5432/crates"

# Redis
redis_url = "redis://localhost:6379"

# Data Import
update_source_data = true
use_only_first_x_crates = 0          # 0 = all crates
max_crates_downloads = 0            # 0 = no filter
insert_crates_in_chunks = 4000

# LLM Configuration
llm_enabled = false
lm_studio_url = "http://localhost:1234/v1"
lm_studio_model = "qwen/qwen2.5-coder-14b"
llm_max_tokens = 4096
llm_temperature = 0.1
llm_max_context_chars = 24576
llm_malicious_score_threshold = 75
```


Appendix C

Web Dashboard

The web dashboard is implemented as a separate Rust crate in the directory (`webpanel-sailfish/`), it is using Axum and Sailfish for server-side HTML rendering. It connects to the same PostgreSQL database and provides an interface for the results. All routes return server-rendered HTML and have support for pagination, filtering, and have sortable columns.

Table C.1: Web dashboard routes.

Route	Method	Description
/	GET	Dashboard: stats, risk score distribution, build.rs flags, top downloads, gitleaks rule frequency
/crates	GET	Paginated crate list. Filters: search, min_downloads, max_downloads. Sort: name, downloads, created_at, updated_at
/dependencies	GET	Dependency list with crate names. Filters: crate_name, crate_id, dependency_id
/vulnerabilities	GET	Cargo-audit results. Filters: crate_name, crate_id, rustsec_id, min_severity
/secrets	GET	Gitleaks results. Filters: crate_name, crate_id, rule_id, min_entropy
/typosquat	GET	Typosquat pairs with all scores. Filters: crate_name, min_combined_score
/scan-results	GET	Aggregated scan results with build.rs flags. Filters: crate_name, min_llm_score, has_vulnerabilities, build_rs_network_calls, build_rs_process_spawning
/metrics	GET	Per crate timing benchmark. Filters: crate_name, crate_id. Displays: total, download, cargo-audit, gitleaks, build.rs, LLM

Appendix D

Artifact Documentation and Reproducibility

D.1 Software Artifacts

Artifact	Description	Location
crates_downloader	Main analysis framework (Rust)	crates_downloader/
webpanel	Dashboard web application (Rust + Axum)	webpanel/
Database migrations	PostgreSQL schema definitions	migrations/
Docker configuration	Container orchestration	docker-compose.*.yaml, D
Configuration template	Analysis parameters	config.toml

D.2 Data Artifacts

Artifact	Description	Format
crates.io dump	Input data from crates.io	CSV files
Analysis results	Complete ecosystem scan	PostgreSQL database dump
Vulnerability findings	cargo audit results	JSON / PostgreSQL
Secret detections	Gitleaks findings	JSON / PostgreSQL
Typosquatting pairs	Similarity computations	PostgreSQL

D.3 Hardware Requirements

Component	Minimum	Recommended
CPU	4 cores	16+ cores
RAM	16 GB	64 GB
Storage	500 GB SSD	1 TB NVMe + tmpfs
GPU	None (CPU inference)	NVIDIA RTX 5000 series for fast inference
Network	100 Mbps	1 Gbps

D.4 Software Requirements

Software	Version	Purpose
Rust	(stable)	Core application
PostgreSQL	17+	Data storage
Redis	7+	Work queue
Docker	24+	Containerization
Docker Compose	2.20+	Orchestration
Gitleaks	8+	Secret detection
LM Studio	0.2.20+	Local LLM hosting
Git	2.40+	Repository cloning

D.5 Reproduction Steps

1. Clone the repository:

```
git clone https://github.com/[REDACTED]/crates-security-framework.git
cd crates-security-framework
```

2. Configure environment:

```
cp config.toml.example config.toml
# Edit config.toml with database and Redis URLs
```

3. Start infrastructure:

```
docker-compose up -d postgres redis
```

4. Initialize database:

```
diesel migration run
```

5. Download crates.io database dump:

```
cargo run --release -- update-database
```

6. Build dependency graph:

```
cargo run --release -- run-analysis
```

7. Start workers (adjust count as needed):

```
docker-compose -f docker-compose.workers.yml up -d --scale worker=10
```

8. Monitor progress:

```
cargo run --release -- queue-status
```

9. Run post-processing:

```
cargo run --release -- compute-dependencies
```

10. Run typosquatting analysis:

```
cargo run --release -- typosquat --min-score 75 --top-crates 10000
```

11. Start web dashboard:

```
cd webpanel && cargo run --release
```

D.6 Expected Outputs

PostgreSQL database containing:

- All crate metadata from crates.io
- Complete dependency graph
- Scan results for all analyzed crates
- Individual vulnerability findings (cargo_audit_results)
- Individual secret findings (gitleaks_results)
- Typosquatting pair candidates (typosquat_results)

Log files in logs/

Working web dashboard

Appendix E

Glossary

Term	Definition
C2 (Command and Control)	Server infrastructure used by attackers to communicate with compromised systems
CVSS	Common Vulnerability Scoring System, a standardized severity rating for vulnerabilities
DAG	Directed Acyclic Graph, used to model dependency relationships
Entropy	Shannon entropy, a measure of randomness/information density in data
Gitleaks	Open-source tool for detecting secrets in Git repositories
LLM	Large Language Model, an AI model trained on large text corpora
RustSec	The Rust security advisory database
Transitive dependency	Dependencies of dependencies (indirect dependencies)
Typosquatting	Registering names similar to popular packages to deceive users
YARA	Pattern matching tool commonly used for malware detection

Table E.1: Glossary of Terms

Appendix F

Generative AI Tools Usage

An overview of how generative AI tools were used during the research and writing process is presented following table:

AI Tool	Use case
Claude Opus 4.6, DeepSeek V4 Pro	Rephrasing paragraphs for clarity Typo checking and grammar corrections
Claude Opus 4.6, DeepSeek V4 Pro	Assistance with Rust code patterns for better code hygiene

Table F.1: Overview of generative AI tool usage in this thesis.

F.1 Claude Opus 4.6, DeepSeek V4 Pro — Rephrasing, Typo Checking, and Bibliography

Purpose: Claude Opus 4.6 and DeepSeek V4 Pro were used for:

1. **Rephrasing:** Paragraphs written by the author were submitted to the mentioned models with instructions to improve clarity, conciseness, and academic tone while preserving the accuracy of the content.
2. **Typo checking:** After each writing session, selected sections were submitted to different AI models with a prompt requesting identification of spelling errors, grammatical mistakes, and awkward phrasing. All suggested corrections were reviewed and rewritten with modifications by the authors.

Method: The prompts used were variations of:

- “*Improve the clarity and academic tone of the following paragraph while preserving ALL technical details and references: [text]*”
- “*Check the following text for typos, spelling errors, and grammatical mistakes: [text]*”

Verification: All generated output was manually reviewed by the author and modified. For rephrasing, the author compared the suggested modifications against the original to ensure no meaning was altered or technical inaccuracies introduced.

Implementation: Rephrased text was accepted or rejected on a word by word basis, with modifications and checks, nothing is copy-pasted.

F.2 Code Generation Assistance

Purpose: The mentioned models were used to assist with Rust code patterns during the development of the analysis framework.

Method: The author described manually wrote and implemented the entire framework, asked for better code hygiene, better patterns and optimizations and received code suggestions. The author adapted, refactored, and integrated the generated code into the existing codebase following the project’s conventions without copy-pasting or accepting agentic changes.

Implementation: No generated code was used verbatim without understanding and modification.